THE YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project- High Level Design

On

Hardened Cloud Infrastructure for E-commerce data

Secure File Storage & AES-256-GCM Encryption Platform for E-Commerce

Student Name: _____

Industry Mentor:	**Guided by:**

Table of Contents

- **Introduction**
  - Scope of the Document
  - Intended Audience
  - System Overview
  - Assumptions
- **System Design**
  - Application Design
  - Process Flow
  - Information Flow
  - Components Design
  - Key Design Considerations
  - API Catalogue
- **Data Design**
  - Data Model
  - Data Access Mechanism
  - Data Retention Policies
  - Data Migration
- **Interfaces**
- **State and Session Management**
- **Caching**
- **Non-Functional Requirements**
  - Security Aspects
  - Performance Aspects
  - Limitations
  - Future Enhancements
  - Testing Summary
- **References**

1. Introduction

SecureVault is a Django-based, Python-powered secure file storage platform designed for the e-commerce domain. It implements AES-256-GCM (Advanced Encryption Standard - Galois/Counter Mode) symmetric encryption combined with PBKDF2-HMAC-SHA256 key derivation to protect sensitive business files including invoices, customer data, payment records, and vendor contracts stored on the server.

The system directly addresses the critical industry challenge of Data-at-Rest protection — ensuring that even if a server's storage is accessed or compromised, the files within remain completely unreadable to an attacker without the correct encryption password. Files are encrypted before being saved to disk and decrypted only on-demand when the correct password is supplied.

1.1 Scope of the Document

This High Level Design (HLD) document covers the complete architecture of the SecureVault system. It describes:

- The overall system architecture and component interactions.
- The AES-256-GCM encryption and PBKDF2 key derivation workflow.
- Local encrypted file storage using Django's FileField and MEDIA_ROOT.
- Database design, data models, and data access patterns.
- REST API endpoints exposed by the Django application.
- Security, performance, and non-functional requirements.

This document does not cover deployment infrastructure, CI/CD pipelines, or low-level code implementation details — those are addressed in the Low Level Design (LLD) document.

1.2 Intended Audience

- Industry Mentor – To review architecture alignment with professional cloud security standards.
- Faculty Guide – To evaluate the academic depth and technical completeness of the design.
- Development Team – To use as a blueprint for implementation in VS Code using Python and Django.
- Security Reviewers – To assess compliance with OWASP, PCI-DSS, and GDPR data-at-rest encryption requirements.

2. System Overview

SecureVault acts as a security layer between the e-commerce business user and local server storage. It intercepts sensitive files uploaded through the web portal, encrypts them using AES-256-GCM before they are written to disk, and stores only the encrypted ciphertext — making data breaches and unauthorized server access effectively harmless.

Technology Stack

Component	Technology / Details
Backend Framework	Django 4.2 (Python) – MVT architecture, ORM, session

	management, CSRF protection.
Encryption Library	Python cryptography >= 41.0 – AESGCM, PBKDF2HMAC, hashes.SHA256, InvalidTag.
File Storage	Django FileField with MEDIA_ROOT – encrypted .vault blobs stored on local server disk.
Database	SQLite (development) / PostgreSQL (production) – managed via Django ORM.
Frontend	HTML5 + CSS3 – Custom dark industrial UI with Space Mono and Syne Google Fonts.
Authentication	Django built-in Auth – session-based login, password hashing, CSRF middleware.
Key Derivation	PBKDF2-HMAC-SHA256 – 480,000 iterations, 256-bit key output, 256-bit random salt.
Vault File Format	Custom .vault format: MAGIC(8B) + SALT(32B) + NONCE(12B) + CIPHERTEXT(N+16B).

System Boundaries

The system boundary encompasses the Django web application server, the SQLite/PostgreSQL database, and local encrypted file storage under the MEDIA_ROOT directory. External to the system boundary are end-user browsers and the underlying operating system file system.

3. System Design

3.1 Application Design

SecureVault follows Django's Model-View-Template (MVT) architectural pattern with a clear separation of concerns across three layers:

- Presentation Layer – HTML/CSS templates rendered by Django views: login.html, register.html, dashboard.html, upload.html, download.html, confirm_delete.html, and audit_log.html.
- Business Logic Layer – Django Views and encryption.py handle encrypt/decrypt operations, file I/O, session management, and audit logging.
- Data Layer – Django ORM models (EncryptedFile, AuditLog) abstract SQLite/PostgreSQL database access with parameterised queries.

3.2 Process Flow

The following describes the complete end-to-end workflow of SecureVault from file upload to encrypted local storage and back to decrypted retrieval.

Encryption Flow (File Upload):

- Step 1 – User Input: User selects a sensitive file via the web portal drag-and-drop zone and provides an encryption password with confirmation.
- Step 2 – Salt Generation: os.urandom(32) generates a cryptographically secure 256-bit random

salt unique to this file encryption operation.

- Step 3 – Key Derivation: PBKDF2HMAC(SHA256, iterations=480000) stretches the password and salt into a strong 256-bit AES encryption key.
- Step 4 – Nonce Generation: os.urandom(12) generates a 96-bit random nonce (IV) for AES-GCM mode, unique per encryption.
- Step 5 – AES-256-GCM Encrypt: AESGCM(key).encrypt(nonce, plaintext, None) produces ciphertext with an embedded 128-bit GCM authentication tag.
- Step 6 – Pack .vault Blob: Packs into binary format: MAGIC(8B) + SALT(32B) + NONCE(12B) + CIPHERTEXT, saved as a .vault file.
- Step 7 – Save to Disk: Django FileField saves the encrypted .vault blob to MEDIA_ROOT/encrypted/ directory on the server.
- Step 8 – Record Metadata: Django ORM saves an EncryptedFile record with original filename, salt, nonce, and file size. The password is never stored.

Decryption Flow (File Download):

- Step 1 – Password Entry: User navigates to the decrypt page for their file and enters the original encryption password.
- Step 2 – Fetch from Disk: Django reads the .vault blob from MEDIA_ROOT/encrypted/ using the stored file path.
- Step 3 – Unpack Header: Reads MAGIC(8B) to verify format, then extracts SALT(32B) and NONCE(12B) from the blob header.
- Step 4 – Re-derive Key: Same PBKDF2-HMAC-SHA256 process re-derives the identical 256-bit key from password and stored salt.
- Step 5 – AES-256-GCM Decrypt: AESGCM(key).decrypt() verifies the GCM authentication tag, then returns plaintext. A wrong password raises InvalidTag immediately.
- Step 6 – Stream to Browser: Django serves the decrypted bytes as an HTTP attachment download with the original filename restored.

3.3 Information Flow

The following describes the movement of data between system components at each stage:

- Browser to Django View – HTTPS POST with multipart form data containing file bytes and encryption password, protected by TLS in transit.
- Django View to encryption.py – In-memory Python bytes object containing file data and password string, never written to disk in plaintext form.
- encryption.py to Django View – Returns the encrypted .vault blob bytes, salt bytes, and nonce bytes.
- Django View to File System – Writes the encrypted .vault blob to MEDIA_ROOT/encrypted/. Plaintext never touches disk.
- Django View to Database – SQL INSERT of file metadata: original filename, salt, nonce, file size, file path. Password is never included.
- File System to Django View – Reads the encrypted .vault blob from disk on a decrypt request.
- Django View to Browser – HTTP response with Content-Disposition: attachment, streaming decrypted file bytes to the user's browser.

3.4 Components Design

SecureVault is composed of the following four core components:

- vault/encryption.py – Core AES-256-GCM encryption module. Key functions: encrypt_file_to_bytes() encrypts and packs the .vault blob; decrypt_file_from_bytes() unpacks and decrypts; derive_key() handles PBKDF2 key derivation; get_encryption_info() returns algorithm metadata for dashboard display.
- vault/views.py – Django request handlers. Orchestrates the full encrypt-save and read-decrypt pipelines. Handles file I/O, audit logging, InvalidTag error catching, and file streaming responses across six views: register, login, logout, dashboard, upload_file, download_file, delete_file, and audit_log.
- vault/models.py – Django ORM models. EncryptedFile stores all file metadata including original_filename, encrypted_file path, salt (BinaryField), nonce (BinaryField), file_type, file_size, and download_count. AuditLog records every security event with user, action, file reference, IP address, and timestamp.
- vault/templates/ – Seven HTML/CSS templates with dark industrial UI design, including a drag-and-drop upload zone, a password strength indicator on the register page, per-file action buttons on the dashboard, and real-time toast notification messages.

3.5 Key Design Considerations

- Zero Password Storage – Encryption passwords are never persisted in the database, session, logs, or any file. Only the random salt and nonce are stored, which are cryptographically useless without the original password.
- Authenticated Encryption – AES-GCM mode provides both confidentiality (encryption) and integrity (authentication tag). A tampered file or wrong password causes immediate, unambiguous failure via Python's InvalidTag exception.
- Unique Key Per File – Each encryption operation generates a fresh 256-bit random salt and 96-bit random nonce, ensuring two identical files encrypted with the same password produce completely different ciphertext.
- In-Memory Processing – File data is read into memory, encrypted, and written as ciphertext. The plaintext never exists on disk; it only lives in RAM during the brief encryption operation.
- OWASP Compliance – 480,000 PBKDF2 iterations meets the OWASP 2023 recommended minimum for PBKDF2-HMAC-SHA256, making dictionary and brute-force attacks computationally expensive.
- Extensible Storage – The system architecture allows future extension to cloud storage (AWS S3, Azure Blob) without modifying the encryption layer — only the file I/O logic in views.py would change.

4. API Catalogue

SecureVault exposes the following HTTP endpoints through Django URL routing. All endpoints marked Session require an active Django session cookie obtained after login:

Endpoint	Method	Description	Auth
GET /login/	GET/POST	Render login form and authenticate user credentials against Django auth backend.	Public

GET /register/	GET/POST	Render registration form and create a new user account with hashed password.	Public
GET /logout/	GET	Invalidate the current user session server-side and redirect to login page.	Session
GET /dashboard/	GET	Display all encrypted files, statistics (total files, size, downloads), and recent audit events.	Session
POST /upload/	GET/POST	Accept file upload, encrypt with AES-256-GCM using provided password, and save .vault blob to disk.	Session
POST /download/<id>/	GET/POST	Accept password, read .vault blob from disk, decrypt and stream original file as download.	Session
POST /delete/<id>/	GET/POST	Permanently delete the encrypted .vault file from disk and remove the database record.	Session
GET /audit/	GET	Display complete security event audit log for the currently authenticated user.	Session

5. Data Design

5.1 Data Model

SecureVault uses two primary database models managed through Django ORM.

EncryptedFile Model:

- id – Auto-increment integer primary key.
- user (FK) – ForeignKey to Django User model, identifying the owner of the encrypted file.
- original_filename – CharField(255) storing the original filename before encryption (e.g., customer_orders.csv).
- encrypted_file – FileField storing the relative path to the .vault blob under MEDIA_ROOT/encrypted/.
- file_type – CharField with choices: invoice / customer_data / payment_info / contract / report / other.
- file_size – PositiveBigIntegerField recording the original file size in bytes before encryption.
- encryption_algorithm – CharField always stored as 'AES-256-GCM' for audit and display purposes.
- salt – BinaryField(32) storing the 256-bit random salt used for PBKDF2 key derivation for this file.
- nonce – BinaryField(12) storing the 96-bit random GCM nonce used during this encryption operation.
- uploaded_at – DateTimeField automatically set to the current UTC timestamp on record creation.
- description – TextField for optional free-text notes provided by the user about this file.
- download_count – PositiveIntegerField tracking the number of successful decryption and

download events.

AuditLog Model:

- id – Auto-increment integer primary key.
- user (FK) – ForeignKey to Django User, SET_NULL on user deletion to preserve audit history.
- action – CharField with choices: upload / download / delete / login / logout / register / failed_decrypt.
- file (FK) – ForeignKey to EncryptedFile, SET_NULL on file deletion to preserve audit history.
- ip_address – GenericIPAddressField recording the client IP address at the time of the security event.
- timestamp – DateTimeField automatically set to the current UTC timestamp on record creation.
- details – TextField providing a human-readable description of the event (e.g., 'Encrypted orders.csv, 4096 bytes').

5.2 Data Access Mechanism

All database access is performed through Django's ORM, which provides:

- Parameterised queries – All user input is automatically escaped, preventing SQL injection attacks.
- QuerySet API – filter(user=request.user) restricts all queries to the authenticated user's own files only.
- Aggregate functions – Sum('file_size') and Sum('download_count') power the dashboard statistics.
- Ordered querysets – Meta class ordering=['-uploaded_at'] ensures newest files appear first.

Critical rule: all EncryptedFile queries are filtered by user=request.user before any operation, ensuring complete data isolation between users. The salt and nonce fields are BinaryField types and are never exposed in any response.

5.3 Data Retention Policies

- Encrypted Files – Retained indefinitely until the user explicitly deletes. On deletion, the .vault blob is removed from MEDIA_ROOT/encrypted/ and the database record is deleted. This action is permanent and irreversible.
- Audit Logs – Retained indefinitely. Audit log entries are append-only and cannot be deleted via the UI, supporting compliance and forensic investigation requirements.
- User Sessions – Django default session expiry is 2 weeks from last activity, configurable via SESSION_COOKIE_AGE in settings.py.
- Encryption Password – Never retained. Used exclusively in memory during the encrypt or decrypt operation and discarded immediately after. Not logged anywhere.
- Original Plaintext – Never written to disk. Processed in memory only. After the .vault blob is saved, the plaintext bytes are garbage collected.

5.4 Data Migration

- python manage.py makemigrations – Generates migration files from model class changes.

- python manage.py migrate – Applies all pending migrations to the configured database.
- For production PostgreSQL: update the DATABASES dictionary in settings.py, run migrate, and transfer existing SQLite data using Django's dumpdata and loaddata management commands.

6. Interfaces

- User Interface (Web Portal) – Seven responsive HTML/CSS pages rendered server-side by Django templates. Dark industrial design with animated grid background. Key UI elements include a drag-and-drop file upload zone with real-time file info display, a password strength meter on the registration page, per-file Decrypt and Delete action buttons on the dashboard, file type badges (Invoice, Payment Info, etc.), and auto-dismissing toast notification messages.
- Django Admin Interface – Available at /admin/ for superusers. Provides a management UI to view and manage User accounts, EncryptedFile records, and AuditLog entries. Useful for system administration without direct database access.
- Django ORM to SQLite – Internal interface. Django ORM translates Python model operations into SQL queries executed against SQLite (development) or PostgreSQL (production), using the psycopg2 adapter for PostgreSQL connections.
- File System Interface – Django's FileField and default_storage backend handle all file I/O. Encrypted .vault blobs are read and written to the MEDIA_ROOT/encrypted/ directory. File paths are stored as relative paths in the database for portability.

7. State and Session Management

SecureVault uses Django's built-in session framework for stateful user authentication and access control:

- Session Storage – Server-side database (django_session table). Only a session ID cookie is stored in the browser; user data is never held client-side.
- Session Duration – Default 2-week browser session, configurable via SESSION_COOKIE_AGE in settings.py.
- CSRF Protection – Django CsrfViewMiddleware adds a CSRF token to all POST forms, preventing cross-site request forgery attacks.
- Login Required – @login_required decorator is applied to all protected views, redirecting unauthenticated users to /login/.
- Logout – django.contrib.auth.logout() flushes all session data server-side immediately on logout.
- Encryption State – Stateless by design. No encryption keys or passwords are ever stored in the session. Every decrypt operation requires the user to re-enter their password.
- User Isolation – All file queries are filtered by request.user, ensuring users can never access another user's encrypted files.

8. Caching

In the current version, SecureVault does not implement application-level caching. Encrypted file blobs must never be cached in decrypted form for security reasons. The following caching strategy is recommended for future production deployment:

- Database Query Cache – Django per-view caching with cache_page() for dashboard statistics

queries (file count, total size) with a 60-second TTL to reduce database load.

- Static Files – WhiteNoise middleware or a CDN such as Cloudflare for serving CSS, JavaScript, and font assets with long-lived cache headers.
- Session Cache – Migrate Django sessions from database to Redis using django-redis for improved performance under high concurrent user load.
- Encrypted Blobs – Must NOT be cached in any form. Always read fresh from disk to ensure data consistency and prevent stale decryption attempts.
- Template Cache – Django template fragment caching for static UI elements like navigation and footer to reduce rendering overhead.

9. Non-Functional Requirements

9.1 Security Aspects

- Encryption Standard – AES-256-GCM (NIST FIPS 197 compliant). The 256-bit key provides 2^256 possible combinations, computationally infeasible to brute-force with any known technology.
- Key Derivation – PBKDF2-HMAC-SHA256 with 480,000 iterations (OWASP 2023 recommended minimum). Makes dictionary attacks and brute-force against the password computationally expensive.
- Salt Uniqueness – 32-byte (256-bit) cryptographically random salt generated per file using os.urandom(32). Prevents rainbow table attacks and ensures identical files produce different ciphertext.
- Nonce Uniqueness – 12-byte (96-bit) cryptographically random GCM nonce generated per encryption using os.urandom(12). Prevents nonce-reuse attacks which would break GCM security.
- Integrity Protection – 128-bit GCM authentication tag is automatically verified during decryption. Any bit-level tampering of the .vault file or use of the wrong password raises InvalidTag immediately.
- No Password Storage – Encryption passwords are never written to the database, session, log files, or any persistent storage. Zero attack surface for credential theft from stored data.
- Audit Logging – All security events (upload, download, failed_decrypt, login, logout, delete) are logged with user identity, timestamp, IP address, and descriptive details.
- CSRF Protection – Django CsrfViewMiddleware enforces CSRF token validation on all POST endpoints, preventing cross-site request forgery attacks.
- SQL Injection Prevention – Django ORM uses parameterised queries for all database operations, automatically sanitising all user-supplied input.
- Data-at-Rest Protection – All files are stored exclusively as encrypted .vault blobs on disk. Even direct file system access yields only AES-256-GCM ciphertext, which is unreadable without the password.

9.2 Performance Aspects

- PBKDF2 Latency – Approximately 0.3 to 0.5 seconds per encrypt or decrypt operation at 480,000 iterations on modern hardware. Acceptable for file-level operations where security justifies the delay.

- File Size Support – In-memory processing supports files up to Django's DATA_UPLOAD_MAX_MEMORY_SIZE (default 2.5 MB). Extend to 100+ MB by implementing chunked streaming encryption.
- Concurrent Users – Django development server is single-threaded. For production, deploy with Gunicorn (4 workers) and Nginx reverse proxy to support 50+ concurrent users.
- Database Performance – SQLite is suitable for development and small deployments up to approximately 1,000 file records. PostgreSQL is recommended for multi-user production deployment.
- Response Time Target – Target under 2 seconds for encrypt and save of files under 1 MB. Target under 5 seconds for files up to 10 MB on standard server hardware.
- Disk I/O – Each upload writes one .vault file to disk. Each download reads one .vault file. No temporary plaintext files are written at any point during the process.

10. References

Source	Reference
NIST FIPS 197	Advanced Encryption Standard (AES) Specification – https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197.pdf
NIST SP 800-38D	Recommendation for Block Cipher Modes: GCM Mode of Operation – https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf
OWASP Password Storage	Password Storage Cheat Sheet – https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
Python cryptography	Python Cryptography Library Documentation – https://cryptography.io/en/latest/
Django Documentation	Django 4.2 Official Documentation – https://docs.djangoproject.com/en/4.2/
Django File Storage	Django File Storage API Documentation – https://docs.djangoproject.com/en/4.2/topics/files/
PCI-DSS v4.0	Payment Card Industry Data Security Standard – https://www.pcisecuritystandards.org/
GDPR Article 32	Security of Processing – Encryption requirement for personal data – https://gdpr-info.eu/art-32-gdpr/

— End of Document —

SecureVault HLD v2.0 | University Name | 2025-26